김승완 Seungwan Kim

AWS와 쿠버네티스 위에서 배포 자동화와 모니터링을 주로 다룹니다.

메가존클라우드 MSP 솔루션 아키텍트 과정에서 7개월 동안 프로젝트 다섯 개를 진행했습니다. EC2 3-tier로 시작해 서버리스와 온프레미스 쿠버네티스 네트워크를 거쳤고 EKS에서는 팀의 CI/CD와 모니터링을 맡았습니다. 그때 배포는 성공인데 서비스는 5xx를 내는 경우를 여러 번 겪었고 마지막 프로젝트에서는 배포 직후 지표를 보고 실패를 판정하는 파이프라인을 혼자 만들었습니다.

AWS SAA-C03 리눅스마스터 2급 TOEIC 960 한국어, 영어 원어민

seungwankim364@gmail.com GitHub 노션 블로그

프로젝트

기간	프로젝트	인원	맡은 일
2026.05 ~ 07	**Quality Gate**	1인	설계부터 구현까지 전부
2026.05 ~ 07	**G2MPT**	4인	CI/CD, 모니터링
2026.03	**Fullcount**	4인	네트워크 설계, 인프라 구축
2026.02	**Photoshoot Blog**	1인	풀스택, AWS 인프라
2026.01	**MyFoodMap**	3인	AWS 인프라 설계, 구축

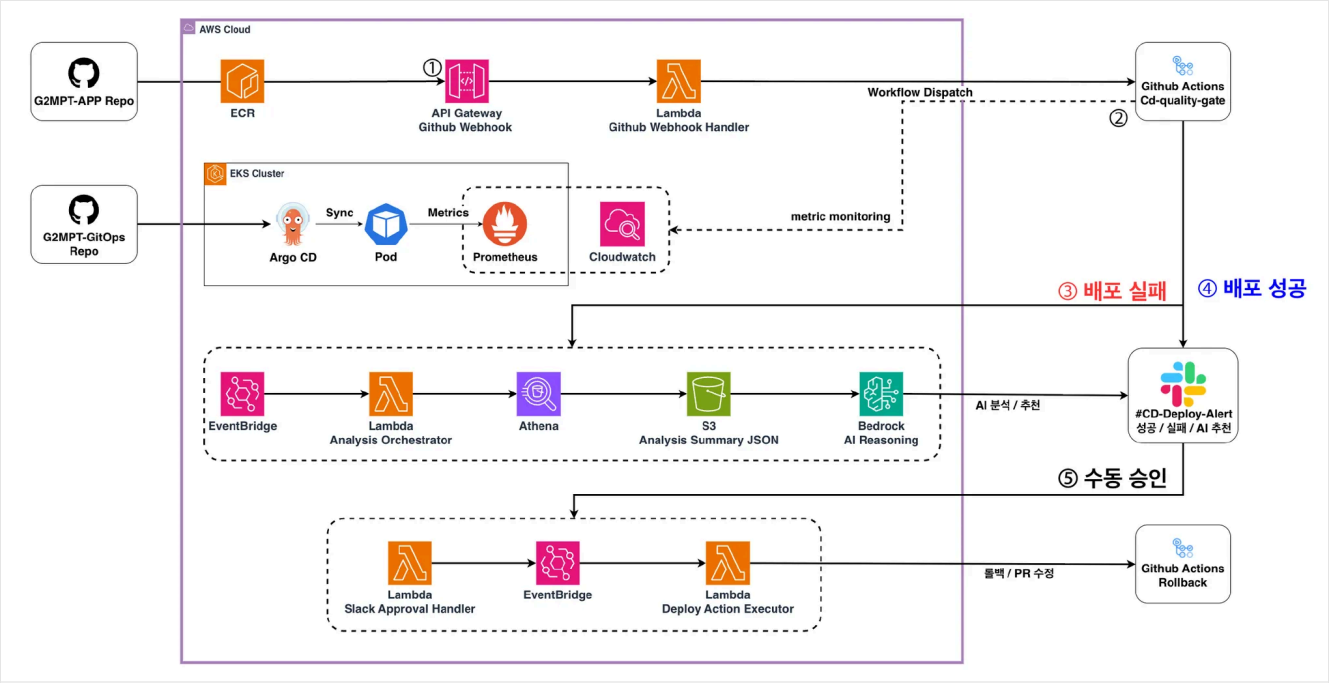
Quality Gate

AI Agent 기반 CD Quality Gate 및 장애 자동 분석 파이프라인

개인 프로젝트 2026.05.11 ~ 07.08

GitHub Actions, Argo CD, Prometheus, CloudWatch, EventBridge, Lambda, Athena, Bedrock, Terraform

G2MPT를 운영하다 보면 GitHub Actions도 성공이고 Argo CD도 Synced인데 서비스는 5xx를 내는 경우가 있었습니다. 파이프라인은 배포 작업이 끝났는지만 확인하고 있었습니다. 그래서 배포가 끝난 뒤 5분 동안 실제 지표를 보고 성공과 실패를 다시 판정하는 단계를 붙였습니다.



배포 후 검증부터 Slack 승인, 롤백까지의 전체 흐름

검증 workflow를 별도 저장소로

검증을 배포 workflow 안에 넣으면 배포가 5분씩 길어지고 검증 쪽이 고장 나면 배포까지 막힙니다. 배포가 끝나면 Webhook이 다른 저장소의 workflow를 호출하도록 떼어 냈습니다.

Prometheus와 CloudWatch를 같이 조회

앱은 정상으로 응답하는데 RDS 커넥션이 바닥나는 경우도 있어서 한쪽 지표만으로는 판정할 수 없습니다. 에러율, 응답 지연, Pod 재시작은 Prometheus에서, RDS와 SQS 상태는 CloudWatch에서 가져옵니다.

롤백은 Slack 승인 후 실행

처음에는 롤백까지 자동으로 하려고 했습니다. 하지만 오탐 한 번에 정상 배포가 되돌아가면 그쪽이 더 큰 사고라서 Bedrock은 원인 후보와 권장 조치까지만 쓰고 실행은 Slack 버튼을 누른 뒤에 돌게 했습니다.

결과

- 배포가 끝나면 5분간 에러율, 응답 지연, Pod 재시작, RDS 커넥션, SQS 적체를 보고 판정
- 실패하면 Athena로 장애 시점 로그를 뽑아 Bedrock 분석 결과를 Grafana, Argo CD 링크와 함께 Slack으로 전송
- Rollback, Manual Fix, Change Apply 세 가지 조치를 승인 버튼 하나로 실행
- Lambda 4개와 EventBridge, Athena, API Gateway를 Terraform으로 관리하며 배포가 없는 동안에는 비용이 거의 발생하지 않음

아쉬운 점

판정이 얼마나 정확한지는 아직 숫자로 확인하지 못했습니다. 정상 배포와 일부러 실패시킨 배포를 반복해 오탐률부터 측정하고 정확도가 확인된 유형만 승인 없이 실행하도록 넓힐 계획입니다.

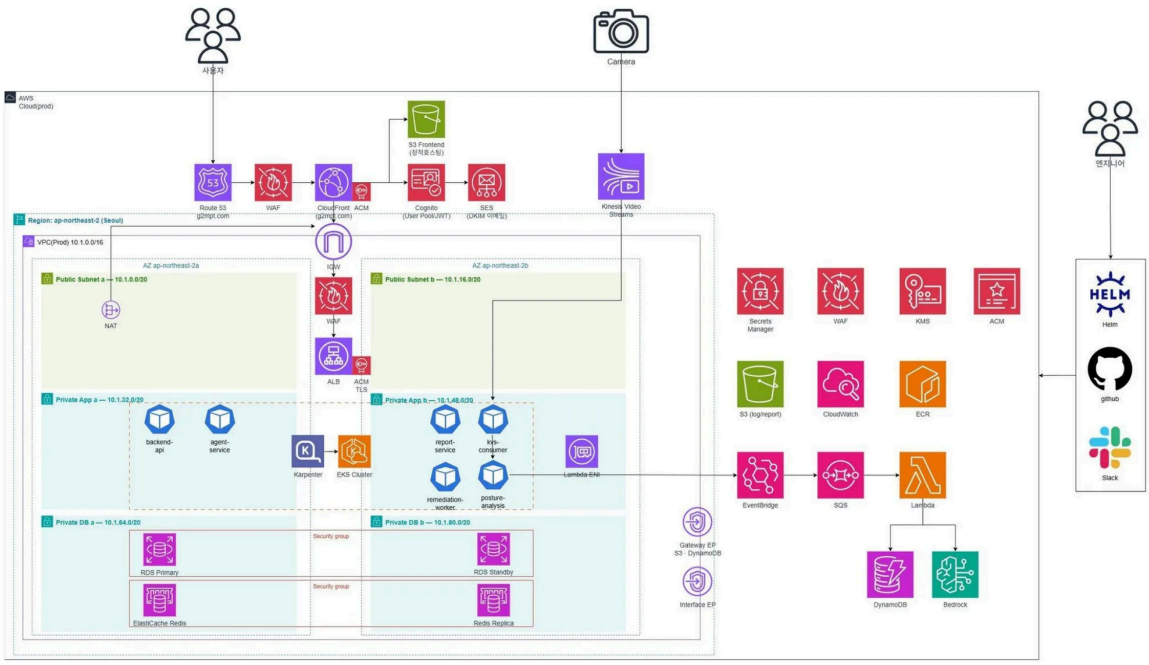
G2MPT

AWS EKS 기반 AI 퍼스널 트레이닝 플랫폼

4인 팀　2026.05.11 ~ 07.08　CI/CD, 모니터링 담당

GitHub Actions, Argo CD, Helm, EKS, Karpenter, Prometheus, Grafana, CloudWatch

사내 서버 한 대에 API, AI, 영상 분석, 리포트가 전부 올라가 있던 서비스를 EKS 마이크로서비스로 옮기는 프로젝트였습니다. 제 몫은 수동 배포와 로그도 지표도 없는 운영을 정리하는 일이었습니다. 서비스가 9개로 쪼개지니 배포만 자동화해서는 각자 다른 모양으로 올라갈 것 같아서 배포 규격과 모니터링을 같이 잡았습니다.



ap-northeast-2 프로덕션 구성

배포는 Argo CD가

GitHub Actions는 이미지를 ECR에 올리고 GitOps 저장소의 image tag만 바꿉니다. 클러스터 반영은 Argo CD가 맡기 때문에 CI가 클러스터 권한을 가질 필요가 없고 AWS 접근도 OIDC로 바꿔 저장소에 남는 액세스 키를 없앴습니다.

Helm 차트 하나로 통일

Deployment, HPA, PDB, ServiceMonitor 등 10종 리소스를 공통 템플릿으로 만들고 서비스마다 values만 다르게 넣었습니다. ServiceMonitor가 템플릿에 들어 있어서 새 서비스를 올리면 Prometheus 수집도 같이 따라옵니다.

알림 경로 두 개

RDS, SQS, ElastiCache, KVS는 클러스터 밖에 있어 Prometheus로는 볼 수 없습니다. Pod와 Node는 Alertmanager로, AWS 리소스는 CloudWatch에서 SNS와 Chatbot을 거쳐 같은 Slack 채널로 보냈습니다.

결과

- 서비스 9개를 각자의 파이프라인으로 배포. 프로젝트 기간 workflow 실행 412회, 배포 1회 평균 2분 30초
- 9개 서비스 모두 GitOps로만 배포. 누가 kubectl로 직접 바꿔도 Argo CD가 Git 상태로 되돌림
- 팀원이 맡은 SQS, KVS까지 포함해 알림 규칙 6개 파일을 한곳에서 관리하고, Alertmanager가 죽은 경우를 잡는 규칙도 따로 둠

배운 점

Helm values의 resource request를 조정하면서 이 값이 클러스터 전체 비용과 묶여 있다는 것을 알았습니다. Karpenter는 request를 보고 노드를 띄우기 때문에 넉넉하게 잡아 두면 쓰지 않는 노드가 계속 늘어납니다. 그 뒤로는 인프라 담당 팀원과 값을 맞춰 가며 조정했습니다.

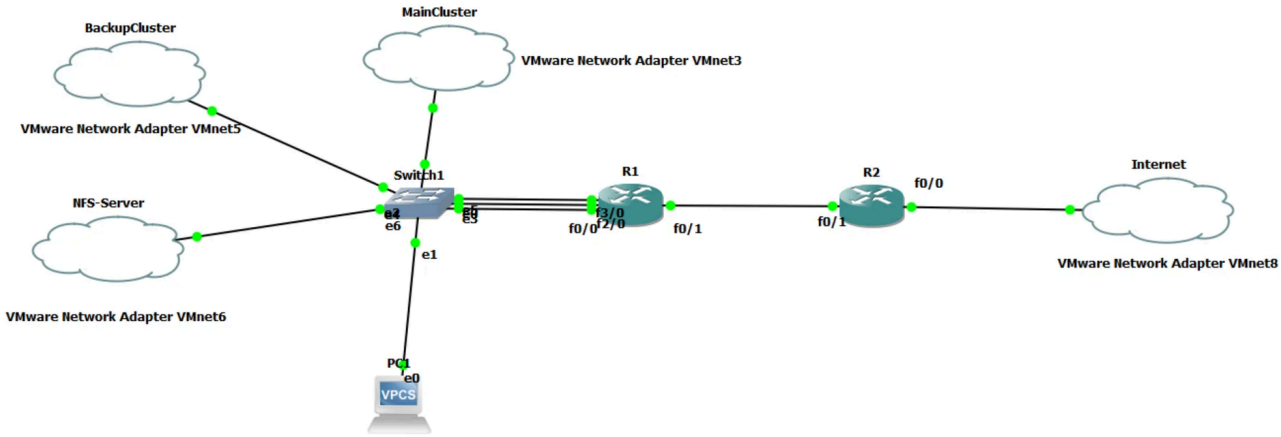
Fullcount

온프레미스 쿠버네티스 기반 MLB 실시간 문자 중계 서비스

4인 팀　2026.03.10 ~ 03.23　네트워크, 인프라 담당

Kubernetes, GNS3, BGP, MetalLB, NAT/PAT, ACL, Tailscale, NFS, cert-manager

AWS라면 ALB 하나로 끝날 일을 라우터부터 직접 구성해야 했습니다. 클라우드가 뒤에서 해 주던 일이 실제로 무엇인지 보고 싶어서 네트워크를 맡았습니다.



GNS3로 구성한 네트워크 토폴로지

BGP로 VIP 광고

Edge, Core, MetalLB를 각각 AS65002, 65001, 65003으로 나누고 MetalLB가 VIP를 eBGP로 광고하게 했습니다. 노드가 하나 죽으면 그 경로만 빠지고 나머지 노드로 트래픽이 갑니다.

서비스 트래픽은 NAT 제외

들어오는 서비스 트래픽까지 NAT을 태우면 클라이언트 IP가 사라지고 경계 라우터 한 대에 부하가 몰립니다. NAT/PAT은 관리용 아웃바운드에만 쓰고 서비스는 VIP 경로로 바로 들어오게 했습니다.

Bastion 대신 Tailscale

Bastion을 두면 그 서버가 또 하나의 공인 IP 노출 지점이 됩니다. 회사망에서 팀원끼리 직접 통신도 막혀 있던 터라 Tailscale로 오버레이를 깔고 VIP 대역은 서브넷 라우팅으로 열었습니다.

결과

- 네트워크 대역 6개, 노드 9대, 라우터 2대, AS 3개로 구성
- Core 라우터에서 Main 클러스터 인터페이스를 내렸을 때 BGP 경로가 빠지고 Backup 클러스터로 접속되는 것까지 확인
- 구축 중 막힌 9건을 계층별로 기록. ACL에 DNS 53번 포트 누락, Tailscale 서브넷 광고 누락, Flannel 미적용 등

배운 점

ACL 한 줄이 빠져서 DNS가 안 됐고, 서브넷 광고 설정 하나 때문에 VIP에 접속이 안 됐습니다.
클라우드에서는 보안 그룹 규칙 한 줄이던 것이 어느 계층에서 무엇을 막고 있었는지 이때 처음 직접 봤습니다. 지금은 네트워크 문제가 생기면 어느 계층에서 끊겼는지부터 좁혀 봅니다.

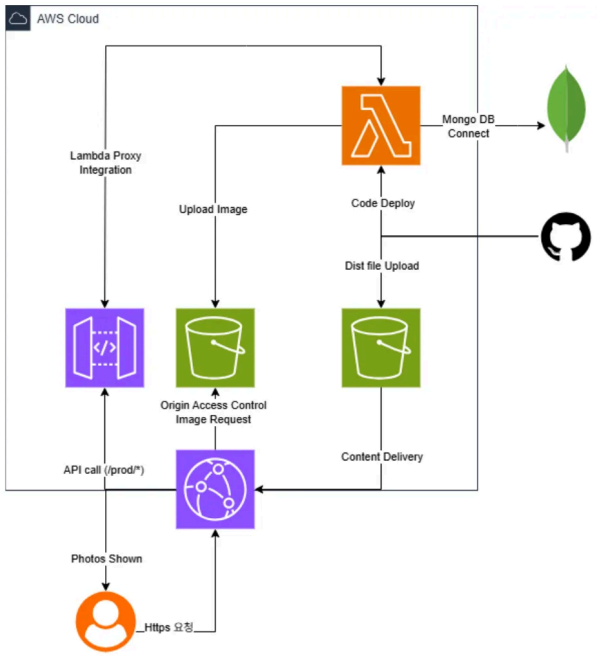
Photoshoot Blog

AWS 서버리스 기반 개인 사진첩 블로그

개인 프로젝트 2026.02.10 ~ 02.20

S3, CloudFront, API Gateway, Lambda, MongoDB Atlas, React, GitHub Actions

직전 MyFoodMap에서는 접속이 없어도 t3.micro 한 대에 매달 10달러 넘게 나갔습니다. 가끔 들어가는 사진첩에 서버를 켜 둘 이유가 없어서 이번에는 서버를 하나도 두지 않고 만들었습니다.



CloudFront 하나로 정적 파일, API, 이미지를 나눠 보냄

CloudFront 하나로 진입

정적 파일, /prod/* API, 이미지를 CloudFront에서 경로로 나눴습니다. 오리진이 하나라 CORS를 따로 설정할 필요가 없었고, 두 S3 버킷은 OAC를 걸어 버킷 주소로 바로 들어오지 못하게 했습니다.

DB 커넥션 재사용

Lambda는 요청이 몰리면 인스턴스가 늘어나는데 그때마다 핸들러 안에서 커넥션을 새로 열면 Atlas 연결 한도에 금방 닿습니다. 커넥션을 핸들러 밖으로 빼서 warm 상태일 때는 다시 쓰게 했습니다.

프론트와 Lambda 배포 분리

화면만 고쳤는데 백엔드까지 다시 배포될 필요는 없어서 파이프라인을 둘로 나눴습니다. 프론트 쪽 마지막에 CloudFront 캐시 무효화를 넣어 바뀐 화면이 바로 보이게 했습니다.

결과

- 상시 실행되는 서버가 없어 요청이 없으면 비용이 거의 발생하지 않음
- 회원가입, JWT 로그인, 사진 업로드와 수정, 삭제, 날짜순 정렬까지 11일 동안 혼자 구현하고 배포
- 프론트와 Lambda 모두 GitHub Actions로 배포

다시 만든다면

이미지 업로드가 아직 Lambda를 거칩니다. API Gateway 페이로드 제한과 Lambda 실행 비용을 생각하면 Presigned URL로 브라우저에서 S3에 바로 올리는 방식이 맞고 다음에는 업로드 경로부터 분리하겠습니다.

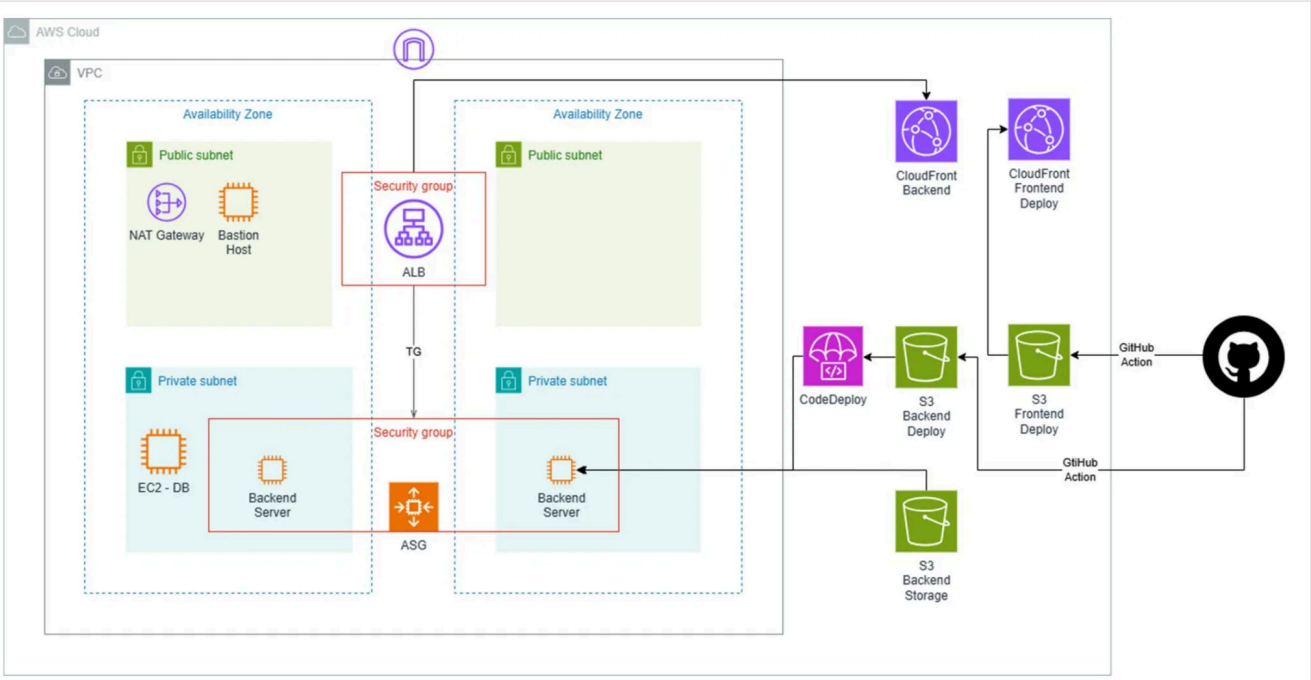
MyFoodMap

AWS 3-tier 맛집 기록 웹 서비스

3인 팀 2026.01.26 ~ 01.29 AWS 인프라 담당

S3, CloudFront, ALB, ASG, CodeDeploy, React, Node.js, MySQL

지도에 내가 간 맛집과 리뷰를 쌓아 두는 서비스입니다. 처음에는 프론트, 백엔드, DB가 EC2 한 대씩이라 어느 하나만 죽어도 서비스가 멈췄고 배포도 손으로 했습니다.



ALB와 ASG, 서브넷 분리를 적용한 뒤의 구성

프론트를 S3로

EC2와 Nginx로 서빙하던 프론트를 S3와 CloudFront로 옮기고 OAC와 HTTPS를 붙였습니다.

백엔드를 2개 AZ로

백엔드를 ALB 뒤 Auto Scaling Group으로 두 AZ에 나눠 두고, Public과 Private 서브넷을 갈라 앱과 DB에는 밖에서 바로 들어올 수 없게 했습니다.

이미지를 S3로

리뷰 사진이 EC2 디스크에 있으면 새로 뜬 인스턴스에서는 그 사진이 안 보입니다. ASG를 붙이려면 이미지부터 S3로 빼야 했습니다.

결과

- 프론트는 GitHub Actions에서 S3와 CloudFront로, 백엔드는 S3와 CodeDeploy로 자동 배포해 수동 배포를 없앰
- 백엔드 인스턴스 하나가 죽어도 다른 AZ에서 계속 응답
- 4일 동안 초기 구성부터 개선 구성까지 진행

배운 점

ASG를 붙이고 나서야 이미지를 왜 S3에 둬야 하는지 알았습니다. 그 뒤 프로젝트부터는 상태를 어디에 둘지를 먼저 정하고 서버 구성을 짭니다. DB는 아직 EC2 한 대에 있어서 RDS Multi-AZ로 옮기는 것이 남은 과제입니다.

경력과 교육

2025.12 ~ 2026.07

메가존클라우드 MSP 솔루션 아키텍트 교육과정

- Linux, 네트워크 기초와 AWS 기반 클라우드 아키텍처 설계
- Docker, Kubernetes 기반 컨테이너 운영과 Terraform IaC
- GitHub Actions, Jenkins CI/CD와 Prometheus, Grafana 모니터링
- 위 다섯 개 프로젝트 수행

2024.02 ~ 2025.08

대한민국 육군, K-9 자주포 포반장

수도기계화보병사단 / 해외 영주권자 신분으로 자진 입대

- 포반원의 안전과 사격 임무 수행을 책임지는 포반장
- 대대 실사격 최고 점수 기록, 전 사격 임무 무사고 완수

2023.09 ~ 2024.02

시지바이오

인체조직 품질관리, SOP 운영, 해외영업

- 해외 인체조직 공급업체와의 미팅 주도, 신규 공급업체 계약 체결
- SOP 기준 검수와 제조 프로세스 확인, SOP 문서 검토와 한영 통번역
- 외부 감사 통역 대응과 시정조치(CAPA) 작성

2016.09 ~ 2023.04

University of Waterloo

Honours Science 졸업 / Waterloo, ON, Canada

- 2020.05 ~ 2023.08 초중고 학생 대상 영어 과외. 학생마다 강약점을 분석해 발음부터 에세이 첨삭까지 맞춤 지도

자격증과 수상

2026.10.02

리눅스마스터 2급

한국정보통신진흥협회

2026.09.04

AWS Certified Solutions Architect, Associate

Amazon Web Services

2025.08.04

병역이행 명예증서

병무청 / 해외 영주권자 신분으로 자진 입대해 받은 표창

2024.05

TOEIC 960

2023.05	**Introduction to Python 3, Introduction to SQL** WatSPEED, University of Waterloo

기술

AWS	VPC, EKS, ECR, S3, CloudFront, ALB/ASG, Lambda, API Gateway, EventBridge, Athena, Bedrock, CodeDeploy 5개 프로젝트 전반
Kubernetes	EKS와 온프레미스 클러스터 운영, Helm 차트 표준화(10종 리소스), HPA, PDB, NetworkPolicy, ExternalSecret G2MPT, Fullcount
CI/CD	GitHub Actions 파이프라인 9종 설계, Argo CD GitOps, OIDC 임시 자격증명, CodeDeploy G2MPT, Quality Gate, MyFoodMap
모니터링	Prometheus 알림 규칙 설계, Grafana 대시보드, Alertmanager, CloudWatch와 Athena 로그 분석 G2MPT, Quality Gate
네트워크	BGP/MetalLB L3 로드밸런싱, NAT/PAT와 ACL 보안 정책, GNS3 라우터 구성, Tailscale Fullcount
IaC	Terraform 기반 AWS 서버리스 인프라 코드화 Quality Gate
Linux	온프레미스 쿠버네티스 호스트 구성과 운영, 패키지 관리, NFS 스토리지 리눅스마스터 2급
기타	Docker, Jenkins, NFS, cert-manager, MongoDB Atlas, MySQL/Prisma, Redis 전반
언어	한국어 원어민, 영어 원어민(TOEIC 960), 프랑스어 중급

연락처

seungwankim364@gmail.com github.com/seungwankim364 노션 블로그